

NPAP: Network Partitioning and Aggregation Package for Python

Marco Anarmo^{1,2}, Benjamin Stöckl^{1,2}, Yannick Werner^{1,2}, and Sonja Wogrin^{1,2}

¹ Institute of Electricity Economics and Energy Innovation (IEE), Graz University of Technology, Inffeldgasse 18, Graz, Austria ² Research Center ENERGETIC, Graz University of Technology, Rechbauerstraße 12, Graz, Austria

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- [Review](#)
- [Repository](#)
- [Archive](#)

Editor: [Vincent Knight](#)

Reviewers:

- [@fneum](#)
- [@FBumann](#)
- [@trintlermint](#)
- [@amcandio](#)
- [@abhishektiwari](#)

Submitted: 10 April 2026

Published: unpublished

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](https://creativecommons.org/licenses/by/4.0/)).

Summary

NPAP (Network Partitioning and Aggregation Package) is an open-source Python library for reducing the spatial complexity of network graphs. Built on NetworkX (Hagberg et al., 2008), it provides an accessible standalone package designed to be readily integrated with other software and frameworks. Instead of treating the spatial reduction process as a single action, NPAP explicitly splits it into two distinct steps: partitioning, which assigns vertices (nodes) to groups (clusters), and aggregation, which reduces the network based on a given assignment. NPAP's strategy pattern architecture allows users to employ and register custom partitioning and aggregation strategies seamlessly without modifying the core code. Currently, NPAP provides 13 different partitioning strategies and two pre-defined aggregation profiles. Although initially developed with a focus on power systems, its architecture is general-purpose and applicable to any network graph.

Statement of need

Real-world electricity grids have grown significantly in size and complexity, potentially spanning many thousands of nodes and edges. Consequently, energy system optimization models representing those networks have become computationally intractable and challenging to solve. To regain tractability and computational efficiency, modelers frequently apply temporal and spatial aggregation techniques (Kotzur et al., 2021). Today, temporal complexity reduction, known as time series aggregation, is a well-established field (Hoffmann et al., 2020; Teichgraber & Brandt, 2022; Wogrin, 2023). Tools like tsam (Time Series Aggregation Module) (Hoffmann et al., 2022) have consolidated multiple temporal aggregation algorithms into a single, reusable Python library that has become a standard component in many energy system optimization modeling frameworks. Before tsam, modelers implemented these methods individually and ad hoc for each project, challenging reusability and comparability.

This is exactly where the research on spatial complexity reduction is at: While there is some research on network partitioning (Frysztacki et al., 2022) and aggregation (Colonetti, 2025) methods available, there exists no standalone tool that brings all methods together and is easy to use, extend, and does not rely on framework-specific data structures. NPAP fills this gap as a standalone package that works with any NetworkX (Hagberg et al., 2008) graph. The target audience includes power systems researchers, energy modelers, network analysts, and more broadly, anyone interested in reducing the spatial complexity of a graph.

State of the field

Within the energy system optimization community, existing implementations of network partitioning and aggregation methods are usually tightly rooted to the respective frameworks' internal data structure and cannot be reused independently or by other tools and frameworks. The built-in spatial clustering module of PyPSA (Brown et al., 2018), for example, provides busmap-based spatial aggregation using methods such as k-means or hierarchical clustering on geographical coordinates of buses, based on PyPSA's Network object and buses DataFrame. As another example, ETHOS.FINE (Klütz et al., 2025) implements spatial reduction through string-, distance-, and parameter-based partitioning modes (Patil et al., 2022), which are strictly tied to the framework's internal region-based data model. Researchers working with other frameworks or custom network representations cannot leverage these implementations without significant adaptation to the core code structure.

The analogy with temporal complexity reduction and the tsam package (Hoffmann et al., 2022) is instructive. NPAP extends this paradigm to the spatial dimension. Its unique contributions include:

- Standalone Architecture: A pip-installable library decoupled from any specific energy system modeling framework.
- Extensibility: A strategy pattern architecture that allows new functionalities to be added without modifying core code.
- Pipeline Control: An explicit conceptual separation between partitioning and aggregation, enabling rigorous control over each stage of the reduction pipeline.
- Advanced Physical Constraints: Built-in support for voltage-aware and AC-island-aware partitioning for real-world electricity networks.

Pipeline architecture

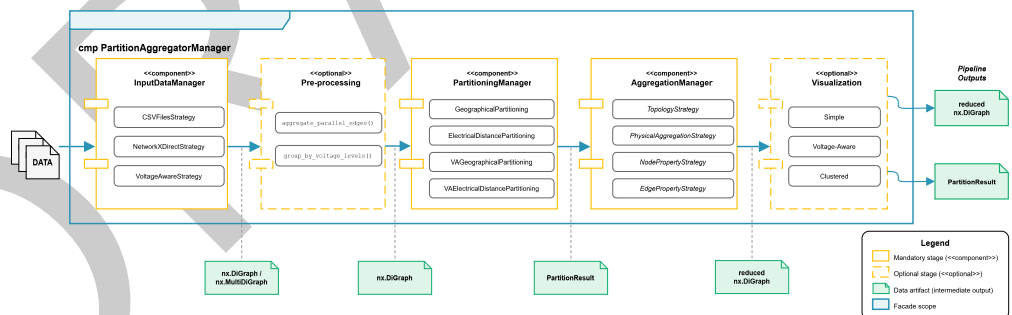


Figure 1: NPAP pipeline architecture.

The full NPAP pipeline is shown in Figure 1. Initially, NPAP performs two stages, preparing the network graph. In the first stage, data is loaded and a NetworkX graph — used for the reduction process — is created and validated. In the second stage, NPAP provides optional pre-processing steps that prepare the network graph for the reduction process. The network can be illustrated throughout the pipeline using the visualization component.

A key design decision in NPAP is the explicit separation of the network reduction process into two stages:

- Partitioning: A partitioning strategy maps each node in the original network to a node in the aggregated network. This step only determines group “membership”; it does not modify the graph itself.

73 2. Aggregation: An aggregation strategy reduces the network topology based on a given
74 partitioning result (mapping) by aggregating nodes, edges, and their associated properties.
75 This separation was identified early in the design process as a fundamental distinction. It
76 gives users fine-grained control: they can swap partitioning algorithms independently of the
77 aggregation method, apply different aggregation strategies to the same partitioning result,
78 compose custom pipelines that combine strategies from different domains, or simply use any
79 of them in isolation. The next section explains the software design in more detail.

80 Software design

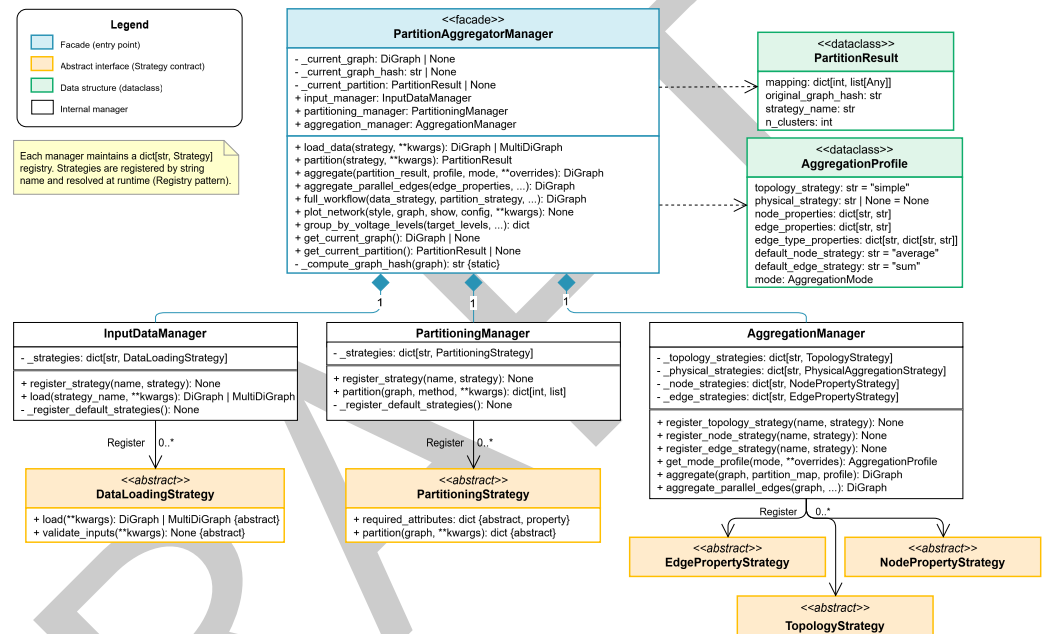


Figure 2: NPAP general class diagram.

81 A broad overview of NPAP's design is shown in Figure 2. The whole workflow is orchestrated
82 and accessed by the PartitionAggregatorManager. NPAP follows a strategy pattern with four
83 categories — data loading, partitioning, topology aggregation, and property aggregation — all
84 orchestrated by three manager classes, shown at the bottom of the Figure. New strategies
85 inherit from abstract base classes and register with their respective managers, enabling users
86 to seamlessly add custom strategies without modifying core code. To reduce entry barriers for
87 contributing to NPAP, the whole library is built on well-established Python packages, such as
88 NetworkX (Hagberg et al., 2008) for graph representation, NumPy (Harris et al., 2020), SciPy
89 (Virtanen et al., 2020), and scikit-learn (Pedregosa et al., 2011) for numerical computation,
90 and Plotly (Inc., 2015) for interactive map-based visualization of results. In the following
91 sections, we introduce the loading and pre-processing of input data, partitioning, and the
92 aggregation processes in more detail.

93 Data loading and pre-processing

94 As NPAP is designed to work with NetworkX graphs, they can be directly passed as input
95 data, facilitating an easy integration into other frameworks or software. Besides, NPAP
96 currently supports importing data from CSV files with two main strategies. The first one
97 works with general node and edge data targeting generic graph structures. The second one is
98 a domain-dependent strategy focused on power grids and includes buses, lines, transformers,

99 converters, and DC links. User-specific data loading strategies can be registered straightforward
100 as shown in Figure 2. Afterward, optional pre-processing steps are carried out through the
101 PartitionAggregatorManager, such as the aggregation of parallel edges. For power grids,
102 voltage-level grouping separates the network into independent sub-graphs per voltage level,
103 enabling voltage-aware partitioning strategies.

104 Partitioning strategies

105 NPAP currently provides four families of partitioning strategies combining geographical and
106 electrical node distance with the option of partitioning voltage levels independently (voltage-
107 awareness). Each family supports 13 different partitioning strategies, such as k-means (Lloyd,
108 1982), k-medoids (Schubert & Rousseeuw, 2021), DBSCAN (Ester et al., 1996), HDBSCAN
109 (Campello et al., 2013), and hierarchical clustering (Ward Jr, 1963). Geographical distance
110 captures latitudes and longitudes and is suitable for any geo-referenced network. Electrical
111 distance computes Power Transfer Distribution Factors (PTDFs) (Wood et al., 2012) to capture
112 the electrical behavior of a power grid rather than its geographical topology.

113 In power grids, NPAP automatically detects alternating current (AC) islands linked solely by DC
114 interconnections and partitions them independently. This, and the voltage-aware partitioning,
115 is achieved by setting the distance matrix entries of nodes in different AC islands and voltage
116 levels, respectively, to infinity. Both approaches are algorithm-agnostic, i.e., they work with
117 any distance-based partitioning method without requiring modifications to the algorithm itself.
118 The partitioning outcome, along with other metadata, is then stored in the PartitionResult.

119 Three-tier aggregation strategy pipeline

120 The aggregation process is decomposed into three sequential steps. The first (topology) tier
121 builds a new network graph based on the mapping result of the partitioning process by creating
122 a representative node for each cluster and adjusting the edges between them accordingly.
123 Afterward, to cover use cases we do not foresee and usability outside the power system domain,
124 we have included an optional second (domain-specific) tier. There, users can modify physical
125 properties of the network graph, e.g., adding new edges that did not exist in the original
126 graph or explicitly modifying certain properties, such as line reactances. In the third (property
127 aggregation) tier, user-specified functions, e.g., sum, average, or equivalent reactances, are
128 then applied to the remaining node and edge properties. These property aggregation strategies
129 can be configured by the user in detail through the aggregation profiles shown in Figure 2. At
130 the moment, NPAP provides two pre-defined aggregation profiles.

131 Research impact statement

132 NPAP is designed as a standalone, pip-installable open-source Python package, released under
133 the MIT license. The library is fully documented on ReadTheDocs and includes an automated
134 test suite with continuous integration across Python 3.10, 3.11, and 3.12. It has been tested
135 on small- and large-scale networks with up to a few thousand nodes.

136 One potential application of NPAP is within energy system modeling frameworks. For illustration
137 purposes, we have integrated it into the well-established PyPSA-Eur (Hörsch et al., 2018)
138 framework through an open pull request introducing NPAP as an alternative to PyPSA's
139 native spatial clustering backend.¹ Thereby showcasing NPAP's framework-agnostic design
140 and highlighting the minimal adoption effort.

141 We further utilized this implementation to demonstrate NPAP's usability and scalability by
142 applying it to the full pan-European transmission network (Xiong et al., 2025), comprising
143 around 6800 nodes and 17500 edges. Leveraging the voltage-aware partitioning strategy

¹<https://github.com/PyPSA/PyPSA/pull/1568>

with geographical distance and k-means clustering (Lloyd, 1982), we analyze the required infrastructure investments for power lines on different voltage levels and transformers using PyPSA (Brown et al., 2018). The work has been submitted to the International Conference on the European Energy Market 2026.

Active development directions include the extension of the available partitioning strategies, e.g., the Adjacent Node Agglomerative Clustering algorithm (Stöckl et al., 2025), and aggregation strategies, e.g., based on PTDFs (Fortenbacher et al., 2018), as well as the extension to other physical infrastructures, such as hydrogen networks, which require distinct strategies. Future extensions include, among others, a filtering module that allows selecting subsets of the network, e.g., a country or region, as an additional pre-processing step before partitioning.

AI Usage Disclosure

Several artificial intelligence (AI) tools were used during the development of NPAP. Google Gemini (Pro) was used for non-code-related tasks, such as learning about power systems. Anthropic Claude (Sonnet 4.5, Opus 4.5, and 4.6), accessed through both the desktop application and the Claude Code terminal interface, was used for code-related activities, including software design brainstorming, implementation support, test coverage, or writing the documentation. Grammarly was used for grammar and spell checking. No other code-generation tools (such as GitHub Copilot, Codex, or Cursor) were used. All AI-generated output was carefully reviewed, tested, and frequently modified before inclusion. The authors take full responsibility for the code.

Acknowledgements

The work done on the package by the contributors of the Institute of Electricity Economics and Energy Innovation (IEE) at Graz University of Technology was funded by the European Union (ERC, NetZero-Opt, 101116212). Views and opinions expressed are, however, those of the author(s) only and do not necessarily reflect those of the European Union or the European Research Council. Neither the European Union nor the granting authority can be held responsible for them.

References

- Brown, T., Hörsch, J., & Schlachtberger, D. (2018). PyPSA: Python for power system analysis. *Journal of Open Research Software*, 6(1), 4. <https://doi.org/10.5334/jors.188>
- Campello, R. J. G. B., Moulavi, D., & Sander, J. (2013). Density-based clustering based on hierarchical density estimates. *Advances in Knowledge Discovery and Data Mining*, 160–172. https://doi.org/10.1007/978-3-642-37456-2_14
- Colonetti, B. (2025). Ward reduction in unit-commitment problems. *Electric Power Systems Research*, 240, 111271. <https://doi.org/10.1016/j.epsr.2024.111271>
- Ester, M., Kriegel, H.-P., Sander, J., & Xu, X. (1996). A density-based algorithm for discovering clusters in large spatial databases with noise. *Proceedings of the Second International Conference on Knowledge Discovery and Data Mining (KDD-96)*, 226–231.
- Fortenbacher, P., Demiray, T., & Schaffner, C. (2018). Transmission network reduction method using nonlinear optimization. *2018 Power Systems Computation Conference (PSCC)*, 1–7. <https://doi.org/10.23919/pssc.2018.8442974>
- Frysztacki, M. M., Recht, G., & Brown, T. (2022). A comparison of clustering methods for the spatial reduction of renewable electricity optimisation models of europe. *Energy Inform*, 5(1), 4. <https://doi.org/10.1186/s42162-022-00187-7>

- 188 Hagberg, A. A., Schult, D. A., & Swart, P. J. (2008). Exploring network structure, dynamics,
189 and function using NetworkX. In G. Varoquaux, T. Vaught, & J. Millman (Eds.), *Proceedings*
190 *of the 7th python in science conference* (pp. 11–15).
- 191 Harris, C. R., Millman, K. J., Walt, S. J. van der, Gommers, R., Virtanen, P., Cournapeau, D.,
192 Wieser, E., Taylor, J., Berg, S., Smith, N. J., Kern, R., Picus, M., Hoyer, S., Kerkwijk,
193 M. H. van, Brett, M., Haldane, A., Río, J. F. del, Wiebe, M., Peterson, P., ... Oliphant,
194 T. E. (2020). Array programming with NumPy. *Nature*, 585(7825), 357–362. <https://doi.org/10.1038/s41586-020-2649-2>
195
- 196 Hoffmann, M., Kotzur, L., & Stolten, D. (2022). The pareto-optimal temporal aggregation of
197 energy system models. *Applied Energy*, 315, 119029. <https://doi.org/10.1016/j.apenergy.2022.119029>
198
- 199 Hoffmann, M., Kotzur, L., Stolten, D., & Robinius, M. (2020). A review on time series
200 aggregation methods for energy system models. *Energies*, 13(3), 641. <https://doi.org/10.3390/en13030641>
201
- 202 Hörsch, J., Hofmann, F., Schlachtberger, D., & Brown, T. (2018). PyPSA-Eur: An open
203 optimisation model of the european transmission system. *Energy Strategy Reviews*, 22,
204 207–215. <https://doi.org/10.1016/j.esr.2018.08.012>
- 205 Inc., P. T. (2015). *Collaborative data science*. Plotly Technologies Inc. <https://plot.ly>
- 206 Klütz, T., Knosala, K., Behrens, J., Maier, R., Hoffmann, M., Pflugradt, N., & Stolten, D.
207 (2025). ETHOS.FINE: A framework for integrated energy system assessment. *Journal of*
208 *Open Source Software*, 10(105), 6274. <https://doi.org/10.21105/joss.06274>
- 209 Kotzur, L., Nolting, L., Hoffmann, M., Groß, T., Smolenko, A., Priesmann, J., Büsing, H.,
210 Beer, R., Kullmann, F., Singh, B., Praktikno, A., Stolten, D., & Robinius, M. (2021). A
211 modeler's guide to handle complexity in energy systems optimization. *Advances in Applied*
212 *Energy*, 4, 100063. <https://doi.org/10.1016/j.adapen.2021.100063>
- 213 Lloyd, S. P. (1982). Least squares quantization in PCM. *IEEE Transactions on Information*
214 *Theory*, 28(2), 129–137. <https://doi.org/10.1109/TIT.1982.1056489>
- 215 Patil, S., Kotzur, L., & Stolten, D. (2022). Advanced spatial and technological aggregation
216 scheme for energy system models. *Energies*, 15(24), 9517. <https://doi.org/10.3390/en15249517>
217
- 218 Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M.,
219 Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D.,
220 Brucher, M., Perrot, M., & Duchesnay, E. (2011). Scikit-learn: Machine learning in Python.
221 *Journal of Machine Learning Research*, 12, 2825–2830.
- 222 Schubert, E., & Rousseeuw, P. J. (2021). Fast and eager k-medoids clustering: O(k) runtime
223 improvement of the PAM, CLARA, and CLARANS algorithms. *Information Systems*, 101,
224 101804. <https://doi.org/10.1016/j.is.2021.101804>
- 225 Stöckl, B., Werner, Y., & Wogrin, S. (2025). Congestion-sensitive grid aggregation for
226 DC optimal power flow. *2025 IEEE Kiel PowerTech*, 1–7. <https://doi.org/10.1109/powertech59965.2025.11180585>
227
- 228 Teichgraeber, H., & Brandt, A. R. (2022). Time-series aggregation for the optimization
229 of energy systems: Goals, challenges, approaches, and opportunities. *Renewable and*
230 *Sustainable Energy Reviews*, 157, 111984. <https://doi.org/10.1016/j.rser.2021.111984>
- 231 Virtanen, P., Gommers, R., Oliphant, T. E., Haberland, M., Reddy, T., Cournapeau, D.,
232 Burovski, E., Peterson, P., Weckesser, W., Bright, J., Walt, S. J. van der, Brett, M.,
233 Wilson, J., Millman, K. J., Mayorov, N., Nelson, A. R. J., Jones, E., Kern, R., Larson, E.,
234 ... Vázquez-Baeza, Y. (2020). SciPy 1.0: Fundamental algorithms for scientific computing
235 in python. *Nature Methods*, 17(3), 261–272. <https://doi.org/10.1038/s41592-019-0686-2>

- 236 Ward Jr, J. H. (1963). Hierarchical grouping to optimize an objective function. *Journal of the*
237 *American Statistical Association*, 58(301), 236–244. [https://doi.org/10.1080/01621459.](https://doi.org/10.1080/01621459.1963.10500845)
238 [1963.10500845](https://doi.org/10.1080/01621459.1963.10500845)
- 239 Wogrin, S. (2023). Time series aggregation for optimization: One-size-fits-all? *IEEE*
240 *Transactions on Smart Grid*, 14(3), 2489–2492. [https://doi.org/10.1109/TSG.2023.](https://doi.org/10.1109/TSG.2023.3242467)
241 [3242467](https://doi.org/10.1109/TSG.2023.3242467)
- 242 Wood, A. J., Wollenberg, B. F., & Sheblé, G. B. (2012). *Power generation, operation, and*
243 *control* (3rd ed.). John Wiley & Sons.
- 244 Xiong, B., Fioriti, D., Neumann, F., Riepin, I., & Brown, T. (2025). Modelling the high-
245 voltage grid using open data for europe and beyond. *Scientific Data*, 12, 277. [https:](https://doi.org/10.1038/s41597-025-04550-7)
246 [//doi.org/10.1038/s41597-025-04550-7](https://doi.org/10.1038/s41597-025-04550-7)

DRAFT